ThreadPool 最新提交与本地版本对比总结

对比对象：GitHub 最新提交 `2260321` 与本地上传版本（threadpool.h / threadpool.cpp / test.cpp）· 2026-10-12

GitHub 最新提交是 `2260321`（"Result机制笔记补充 & limit初始化"），代码上只改了 `Semaphore` 的 `limit` 初始化。本地上传的文件比它更新，包含两块还没提交的改动：**Result 改成 `ResultState` 共享状态**，以及**实现了 Cached 模式**。

一、改动总览

方面	GitHub 最新提交	本地上传版本
结果存放	`Result` 自己存 `any_` + `sem_`，持有 `shared_ptr<Task>`	新增 `ResultState`（`any_` + `sem_`），`Result` 只持有它
Task → Result	裸指针 `Result*`	`shared_ptr<ResultState>`
无 Result 的任务	`exec()` 里不执行	加了 `else { run(); }`，照常执行
提交失败	`Result(sp, false)`	`Result(nullptr, false)`
线程容器	`vector<unique_ptr<Thread>>`	`unordered_map<int, unique_ptr<Thread>>`
线程函数	`threadFunc()`	`threadFunc(size_t threadid)`
Cached 模式	无	动态扩容 + 空闲回收

二、改动一：`ResultState`（重点）

新的所有权关系： `Task` 和 `Result` 各持有一个 `shared_ptr<ResultState>`，指向同一份共享状态。

- **上次讲的悬空问题消失了。** 旧版里 `Task` 用裸指针指向 `Result`，临时 `Result` 一析构，任务执行完 `setVal` 就是未定义行为。现在谁先销毁都不影响，状态对象活到最后一个持有者释放为止。`test.cpp` 里三条没接收返回值的 `submitTask` 现在是安全的。
- **不再依赖"Result 生命周期更长"这种约定。** 强弱引用不对称的设计没有了，两边都是共享所有权，也不会形成循环引用，因为 `ResultState` 不反向引用 `Task` 或 `Result`。

- **`Result` 不再持有 `Task`，职责更单一。** 它只负责"取结果"，存结果和等结果交给 `ResultState`。
- **提交失败的路径更干净。** 任务根本没进队列，状态也就没人持有，`get()` 靠 `isValid_ || state_ == nullptr` 直接返回空值。

这一版仍然成立的：

- `Result` 里还有 `atomic_bool`，不可拷贝也不可移动，所以仍然依赖 C++17 的复制省略。
- `get()` 只能调用一次，第二次会永远阻塞。
- `~Task() = default;` 仍然不是虚函数。

三、改动二：Cached 模式（三条 TODO 的落地）

1. **设置接口。** `setMode`、`setTaskQueMaxThreshHold`、`setThreadSizeThreshHold` 都加了 `isPoolRunning_` 判断，启动后不允许再改。
2. **线程 id 与 `unordered_map`。** `Thread` 用静态 `generateId_` 分配 id，`ThreadFunc` 变成 `void(size_t)`，`start()` 把 id 传给线程。这样线程超时退出时才能 `threads_.erase(threadid)` 把自己从容器里删掉，这也是 `vector` 换成 `map` 的原因。
3. **扩容。** `submitTask` 里满足 `taskSize_ > idleThreadSize_ && curThreadSize_ < threadSizeThreshHold_` 就新建线程。
4. **回收。** `threadFunc` 在 Cached 模式下改为 `wait_for(1s)` 循环，用 `lastTime` 记录上次执行完任务的时间。超时后若空闲 ≥ `THREAD_MAX_IDLE_TIME` 且 `curThreadSize_ > initThreadSize_`，就删除自己并 `return`。
5. **计数。** `idleThreadSize_` 在启动和扩容时加一，取任务时减一，执行完再加一。`curThreadSize_` 在扩容和回收时变化，两者都是 `atomic_uint`。

实测结果： 6 个任务提交后输出 2 次 `>>> create new thread <<<`，闲置约 10 秒后有 2 个线程 `exit!`，回到初始 4 个，最终结果是 `45000150000`。